

DT4PCP

IoT-Driven Explainable Digital Twin Framework

for Early Type-2 Diabetes Risk Prediction & Personalized Preventive Care

Full Project Document — Wearable Hardware + Software Stack

1. Project Overview

DT4PCP is a custom wearable IoT device paired with an AI-powered mobile application designed to predict Type-2 Diabetes Mellitus (T2DM) risk at an early stage. The system collects real-time physiological data through wrist-worn sensors, processes it through a machine learning pipeline, and delivers explainable, personalized health insights via a Flutter mobile app.

This is not a medical diagnosis system. It is an early warning and preventive care support tool designed for research, academic, and wellness purposes.

2. Objectives

- Build a fully functional wrist-worn IoT wearable with multiple physiological sensors.
- Develop an ML pipeline using XGBoost and LightGBM for diabetes risk prediction.
- Implement Explainable AI (SHAP + LIME) to make predictions transparent.
- Create a Digital Twin model continuously updated from sensor data.
- Provide Timeline Analytics to track health trends over time.
- Enable What-If Simulation for lifestyle change impact prediction.
- Integrate an LLM-based health assistant (Groq free tier) for personalized guidance.
- Deliver a complete Flutter mobile app connecting all modules.

3. Hardware Design

3.1 Component List

Component	Model	Purpose	Est. Cost (INR)
Microcontroller	ESP32 (38-pin)	BLE + Wi-Fi + processing	■400
HR + SpO2 Sensor	MAX30102	Heart rate & oxygen saturation	■300
Accelerometer / Gyro	MPU6050	Step count & activity tracking	■200
Body Temp Sensor	MLX90614	Skin/wrist temperature	■400
Blood Pressure	HW-887 (indirect)	Estimated BP via PPG	■500
Battery	LiPo 3.7V 500mAh	Portable power supply	■300
Charging Module	TP4056	USB LiPo charging	■50
Enclosure	3D Printed PLA	Wristband casing	■300
		TOTAL	~■2,450

3.2 Wiring Summary

All sensors communicate with ESP32 via I2C protocol (SDA/SCL). The MAX30102 and MLX90614 share the I2C bus on GPIO 21 (SDA) and GPIO 22 (SCL). MPU6050 uses the same bus with a different I2C address. The TP4056 module handles LiPo charging via USB micro port. A power switch is placed between the LiPo and ESP32 VIN pin.

3.3 Enclosure Design

A custom 3D-printed wristband enclosure is designed to house the ESP32, all sensors facing the inner wrist, and the LiPo battery. The form factor is similar to a standard fitness band. Sensor windows are left open for the MAX30102 (light-based) and MLX90614 (infrared). The enclosure uses PLA filament and includes a strap mount.

3.4 Power Estimate

Component	Current Draw	Estimated Battery Life
ESP32 (active BLE)	~240mA	
MAX30102	~0.7mA	
MPU6050	~3.9mA	
MLX90614	~1.5mA	
HW-887	~5mA	
Total	~250mA	~2 hours (500mAh) continuous
With deep sleep cycles	~30mA avg	~15–16 hours

Deep sleep mode will be implemented on the ESP32 — sensors polled every 30 seconds, BLE active only when app is connected.

4. Firmware (ESP32)

4.1 Tech Stack

- Language: Arduino C++ / MicroPython
- IDE: Arduino IDE / VS Code + PlatformIO
- Libraries: Wire.h (I2C), MAX30105 library, MPU6050 library, BLEDevice.h

4.2 Firmware Flow

Step	Action
1	Boot → Initialize I2C bus and all sensors
2	Read HR + SpO2 from MAX30102
3	Read temperature from MLX90614
4	Read accelerometer data from MPU6050 → compute steps
5	Read BP estimate from HW-887
6	Package all readings as JSON payload

7	Broadcast via BLE GATT characteristic
8	Enter deep sleep for 30 seconds → repeat

4.3 BLE Protocol

The ESP32 acts as a BLE peripheral with a custom GATT service. The Flutter app acts as BLE central and connects to read sensor data. Data is sent as a JSON string over a notify characteristic. The app also writes user profile data back to a writable characteristic.

5. Mobile App (Flutter)

5.1 Tech Stack

- Framework: Flutter (Dart)
- BLE: flutter_blue_plus package
- State Management: Riverpod / Provider
- Backend Communication: http / dio package
- Charts: fl_chart
- Database (local): sqflite for offline caching

5.2 App Screens

Screen	Purpose
Home / Dashboard	Live sensor readings, current risk score, quick summary
Risk Prediction	Detailed ML output — Low / Moderate / High with score %
XAI Explanation	SHAP feature importance chart, LIME explanation text
Digital Twin	Visual health avatar updated with latest sensor data
Timeline Analytics	Line charts showing HR, SpO2, risk score over time
What-If Simulator	Sliders for activity, weight, sleep → predicted risk change
LLM Health Assistant	Chat UI for personalized health advice
Profile	User info — age, height, weight, family history, gender
Settings	BLE device pairing, notification preferences, data export

6. Backend (FastAPI)

6.1 Tech Stack

- Framework: FastAPI (Python)
- Database: Supabase (PostgreSQL)
- Hosting: Render / Railway (free tier)
- ML Serving: joblib model loading + prediction endpoint
- XAI: SHAP + LIME computed at prediction time

6.2 API Endpoints

Method	Endpoint	Purpose
POST	/predict	Accept sensor + profile data, return risk score
POST	/explain	Return SHAP values and LIME explanation
POST	/simulate	What-If simulation with custom input values
GET	/timeline/{user_id}	Fetch historical risk data for charts
POST	/twin/update	Update Digital Twin with latest readings
GET	/twin/{user_id}	Fetch current Digital Twin state
POST	/chat	Forward message to LLM assistant, return response
POST	/user/profile	Save or update user profile

7. Machine Learning Module

7.1 Input Features

Feature	Source
Heart Rate (bpm)	MAX30102 sensor
SpO2 (%)	MAX30102 sensor
Step Count (daily)	MPU6050
Activity Level	MPU6050 (derived)
Body Temperature (C)	MLX90614
Blood Pressure (est.)	HW-887
Age	User Profile
BMI	User Profile (height + weight)
Family History	User Profile
Sleep Duration (hrs)	User Profile / manual input

7.2 Algorithms

- XGBoost — primary model, handles missing values well
- LightGBM — secondary model, fast and memory efficient
- Logistic Regression — baseline model for comparison
- Ensemble voting for final prediction

7.3 Output

- Risk Label: Low / Moderate / High
- Risk Score: 0–100%
- Confidence: model prediction probability

7.4 Dataset

Training data sourced from the PIMA Indians Diabetes Dataset and CDC BRFSS dataset (publicly available). After initial training, the model will be fine-tuned with real sensor readings collected from the wearable to reduce domain mismatch.

8. Explainable AI (XAI)

8.1 SHAP

SHAP (SHapley Additive exPlanations) computes feature contribution values for each prediction. A waterfall chart is shown in the app displaying which sensor readings or profile attributes pushed the risk score higher or lower. This builds user trust and transparency.

8.2 LIME

LIME (Local Interpretable Model-agnostic Explanations) generates a local explanation for each prediction by perturbing input features and observing output changes. The Flutter app displays a simplified text explanation such as: 'Your elevated resting heart rate and low activity level are the main contributors to your moderate risk score.'

9. Digital Twin

The Digital Twin is a continuously updated virtual model of the user's health state. It is stored in Supabase and updated every time new sensor data is received. The twin stores:

- Latest vitals snapshot
- 7-day rolling averages
- Risk score history
- Lifestyle trend indicators
- Anomaly flags (e.g., sudden HR spike)

In the Flutter app, the Digital Twin is visualized as a health card showing current status across all parameters with color-coded health indicators (green / yellow / red).

10. Timeline Analytics

All sensor readings and risk scores are time-stamped and stored in Supabase. The Flutter app renders interactive line charts using `fl_chart` showing:

- Heart rate trends (daily / weekly / monthly)
- SpO2 levels over time
- Activity / step count history
- Risk score evolution
- Sleep duration tracking

Users can identify patterns such as rising resting heart rate over weeks or declining activity, which are early warning indicators for T2DM risk progression.

11. What-If Simulation

The What-If Simulator allows users to modify lifestyle parameters using sliders and observe the predicted change in their diabetes risk score in real-time. Parameters available for simulation:

Parameter	Slider Range	Impact
-----------	--------------	--------

Daily Steps	0 – 15,000	Higher steps → lower risk
Sleep Duration	4 – 10 hrs	Optimal 7-8 hrs → lower risk
BMI / Weight	Current ± 20%	Lower BMI → lower risk
Activity Level	Sedentary → Active	Higher activity → lower risk
BP (systolic)	90 – 180 mmHg	Lower BP → lower risk

The simulation calls the /simulate FastAPI endpoint with modified values and returns a new predicted risk score, shown alongside the current score for comparison.

12. LLM-Based Health Assistant

A conversational health assistant is integrated into the Flutter app using the Groq API (free tier — llama3-8b-8192 model). The assistant is given context about the user's current risk score, XAI explanation, and Digital Twin state before each message.

12.1 Capabilities

- Explain why the user has a High / Moderate / Low risk score in simple language
- Suggest personalized lifestyle changes based on sensor data
- Answer questions about diabetes prevention, symptoms, and diet
- Provide motivational nudges when risk is increasing
- Summarize weekly health trends on request

12.2 LLM Stack

Item	Choice	Cost
LLM Provider	Groq API	Free tier
Model	llama3-8b-8192	Free
Context injection	Risk score + XAI + Twin state	—
Rate limit	~30 req/min (free)	Sufficient for demo

13. Full System Architecture

Layer	Component	Technology
Hardware	Wearable sensors + ESP32	Arduino C++ / I2C / BLE
Communication	BLE GATT	flutter_blue_plus
Mobile App	Flutter UI + local cache	Flutter + sqflite
Backend API	REST endpoints	FastAPI (Python)
ML Engine	Risk prediction	XGBoost + LightGBM
XAI Engine	Explainability	SHAP + LIME
Digital Twin	Health model	Python + Supabase

Timeline DB	Historical data	Supabase PostgreSQL
LLM Assistant	Conversational AI	Groq API (free)
Hosting	Backend deployment	Render / Railway (free)

14. Suggested Project Timeline

Phase	Duration	Deliverable
Phase 1 — Hardware	Week 1–2	Sensors wired + ESP32 firmware reading data
Phase 2 — BLE + App Base	Week 3–4	Flutter app receiving live sensor data via BLE
Phase 3 — ML Pipeline	Week 5–6	Trained model + FastAPI prediction endpoint live
Phase 4 — XAI	Week 7	SHAP + LIME integrated in app
Phase 5 — Digital Twin + Timeline	Week 8–9	Twin model + historical charts in app
Phase 6 — What-If + LLM	Week 10–11	Simulator + chat assistant integrated
Phase 7 — Testing + Polish	Week 12	Full integration test + UI polish + documentation

15. Total Cost Summary

Category	Items	Cost
Hardware	ESP32 + sensors + battery + case	~\$2,450
Software	All frameworks and libraries	\$0 (open source)
LLM API	Groq API	\$0 (free tier)
Backend Hosting	Render / Railway	\$0 (free tier)
Database	Supabase	\$0 (free tier)
	TOTAL PROJECT COST	~\$2,450

16. Known Risks & Mitigation

Risk	Mitigation
BP sensor (HW-887) inaccuracy	Label as 'estimated BP' — not clinical value. Use for trend only.
ML model mismatch with real sensor data	Collect 50–100 real readings → fine-tune model on actual data.
BLE connection drops	Implement auto-reconnect logic + local buffer on ESP32.
Battery life too short	Implement deep sleep mode — poll sensors every 30 seconds.
MLX90614 wrist accuracy	Calibrate with reference thermometer, add offset correction.
Groq API rate limit	Queue requests client-side, show 'thinking' indicator to user.

17. Disclaimer

This project is intended for academic research, proof-of-concept demonstration, and wellness awareness purposes only. It is NOT a certified medical device and does NOT replace clinical diagnostic tests including HbA1c, Fasting Blood Glucose, OGTT, or Continuous Glucose Monitoring (CGM). All risk predictions are indicative only. Users should consult a qualified healthcare professional for any medical decisions.